

03-25 Lecture: Cryptographic Primitives

Foundations for Blockchain Systems

Date & Time: 2026-03-25 22:03:11

Location: [Insert Location]

Instructor: Jean Martina

Theme

Cryptographic primitives — including hash functions, digital signatures, randomness, and consensus — are the foundational cornerstones of trustless, decentralized blockchain systems, replacing the need for central authority with mathematically verifiable trust. Blockchain security is built upon the composition of these primitives, each solving a specific problem, and the system's overall security is only as strong as its weakest component. Emerging primitives like zero-knowledge proofs are increasingly critical for privacy-preserving blockchain systems.

Key Points

1. Cryptocurrencies are called “crypto” currencies because they are fundamentally built on cryptography; understanding cryptographic primitives is essential before studying blockchains.
2. Blockchain systems are designed to be “trustless” — not meaning without trust, but meaning trust is placed in mathematics and cryptographic protocols rather than in central authorities or specific entities.
3. The security of roughly 70–80% of the internet today relies on the mathematical hardness of prime number factorization, a concept taught as early as sixth grade.
4. Quantum computers pose an existential threat to current cryptographic standards (e.g., RSA-based systems and elliptic curve cryptography) because they can solve these hard mathematical problems in a single step given sufficient qubits.
5. Five key cryptographic primitives underpin blockchains: cryptographic hash functions (integrity), digital signatures (ownership), randomness (key security and entropy), consensus (distributed agreement), and — increasingly — zero-knowledge proofs (privacy).

6. A cryptographic hash function maps an arbitrarily large input domain to a fixed-size output, making it computationally infeasible to reverse; the three critical security properties are pre-image resistance, second preimage resistance, and collision resistance.
7. The avalanche effect ensures that changing even a single bit in the input produces a drastically different output, with at least 50% of output bits changing unpredictably — a deliberate feature, not a flaw.
8. A blockchain is a linked list where each block contains the hash of the previous block, ensuring structural integrity across the entire chain; changing one bit breaks all subsequent hashes.
9. The Merkle tree is a binary hash tree used in blockchains to organize transactions in leaves, enabling $O(\log n)$ verification instead of $O(n)$ linear search.
10. Authorization in blockchain is achieved through digital signatures (ECDSA), replacing the role of a bank's username/password or physical card; identity is defined solely by possession of a private key.
11. Asymmetric cryptography uses two mathematically linked keys: encrypting with a private key produces a digital signature verifiable by anyone with the corresponding public key, providing authentication, integrity, and non-repudiation.
12. In blockchain terminology, the public key is the "account" and the private key is the "wallet" — ownership is defined entirely by possession of the private key, and losing it means irreversible, permanent loss of funds.
13. ECDSA (Elliptic Curve Digital Signature Algorithm) achieves approximately the same security as RSA with much smaller key sizes (~256 bits vs. ~2048 bits) due to higher primality density (~60%) among curve points.
14. Randomness (entropy) is foundational to cryptographic security; weak or predictable randomness leads to predictable keys and broken systems — seeding PRNGs with low-entropy data (e.g., timestamps) is a critical and historically exploited vulnerability.
15. Quantum computers exist today, but a "cryptographically relevant" quantum computer capable of breaking RSA or elliptic curve cryptography has not yet been achieved; key government transition deadlines are 2027 (post-quantum PKI), 2030 (all government systems), and 2035 (RSA/ECDSA deprecated).
16. Hash functions are more resilient to quantum threats than asymmetric cryptography — the collision space reduces from 2^{256} to 2^{128} , which remains very safe.
17. Proof-of-Work mining involves repeatedly hashing a block with varying nonce values until the resulting hash meets a target condition (required number of leading zeros); the network self-calibrates difficulty to maintain a ~10-minute block interval.

18. Bitcoin transactions are fully public, meaning wallet balances are traceable; zero-knowledge proofs (ZKPs) allow one party to prove a statement is true without revealing the underlying data and are expected to become a mandatory blockchain feature within approximately five years.
19. Blockchain is not a single technology — it is a composition of cryptographic primitives, each solving a distinct problem, and the overall security is only as strong as its weakest component.
20. Most technology underlying blockchains is not entirely new — it is a reinvention and recombination of existing cryptographic and distributed systems concepts applied to a new problem.

Highlights

- "Trustless is that it can be trusted, but I do not need to trust someone. I trust the system; I do not have to have trust in specific entities." -- Jean Martina
- "We do not trust in the central bank anymore when we're dealing with cryptocurrencies. But we shift trust; we trust in mathematics." -- Jean Martina
- "If you have one of these cryptographic primitives failing, then the whole blockchain will fail." -- Jean Martina
- "Most of the technology that is behind here is not new technology; it's just reinvention. It's regluing the technology together in such a way that makes it work for this kind of distributed system." -- Jean Martina
- "Do you believe that it's easier to break SHA-256 or to corrupt someone in a central bank?" -- Jean Martina
- "The number of slices doesn't change the size of the cake." -- Jean Martina
- "Finding collisions having collisions is not the problem. The problem is that if you can create collisions, if you can engineer collisions." -- Jean Martina
- "Avalanche effect is not a bug — it is a feature that we have." -- Jean Martina
- "A blockchain is nothing more than a linked list where you have the hash of the next block — and with that, it is very straightforward." -- Jean Martina
- "Holding only one hash, you can hold integrity of the whole list." -- Jean Martina

- "The hash is like basically chewing the information and spitting out a fixed instance." -- Jean Martina
- "The identity becomes the possession of the private key." -- Jean Martina
- "A wallet is simply a private key. So, that's basically the control that you have on the funds that are on your account." -- Jean Martina
- "If I encrypt something with my private key and you know my public key and you can decrypt, you know that only I could have done that. So that's called a digital signature." -- Jean Martina
- "Non repudiation means this was done by that person, and the person cannot deny that they done that." -- Jean Martina
- "What's your account? 32 bytes. What's your wallet? 32 bytes. What's the hash of the next block? 32 bytes. What's the identifier of the transaction? 32 bytes." -- Jean Martina
- "The race is not having quantum computers. The race is having cryptographically relevant quantum computers." -- Jean Martina
- "Who owns the keys owns the asset." -- Jean Martina
- "Loss of private key means irreversible loss." -- Jean Martina
- "The wallet doesn't hold anything. What holds your money is the blockchain itself." -- Jean Martina
- "Self custody is very, very hard. What I recommend you doing is avoid it." -- Jean Martina
- "Security depends always on the weakest component." -- Jean Martina
- "It doesn't matter if ECDSA is very good, or if any cryptographic system that you are developing is good. If you use bad randomness, then it will break." -- Jean Martina
- "Randomness must be unpredictable, high entropy, and properly seeded." - Jean Martina
- "Blockchain is not a single technology; it's not a single crypto. It's a composition of primitives; each primitive is in there to solve a specific problem." -- Jean Martina
- "With zero-knowledge proof, you can reveal information without having to resort to reveal the data itself." -- Jean Martina
- "Understanding these primitives and how they work is paramount to understanding how blockchains are built in general terms." -- Jean Martina
- "Alan Turing developed everything for one sole purpose: to break cryptography. That was all the theories of Turing, the father of modern

Topics

1. Introduction and Course Context

This lecture series serves as the deepest computer science sessions of the semester, providing cryptographic foundations necessary for understanding blockchains. The content is explicitly paired with a follow-up concepts lecture and deliberately avoids full depth (which would require a dedicated computer security course).

- This is a foundations-of-cryptography lecture, not a blockchain lecture per se; cryptographic primitives must be understood before blockchain structures can be explained.
- The course progression: introductory lecture → blockchain event → cryptographic foundations → concepts lecture.
- A full computer security course covers these same primitives over approximately a month and a half; this lecture covers the essentials in two hours.
- The focus is specifically on cryptographic primitives as they relate to trustless, decentralized systems — not cryptography in its full breadth.

2. The Problem of Trust and Central Authority in Traditional Monetary Systems

Traditional monetary systems depend entirely on trusted central intermediaries — central banks and commercial banks — to issue currency, manage identities, validate transactions, maintain ledgers, and resolve disputes. The value of fiat currency itself is tied to the authority and credibility of these institutions.

- Central banks determine the money supply and the value of currency; inflation occurs when money is issued faster than economic output grows (the “cake” analogy: more slices don’t make a bigger cake).
- Banks serve as the authoritative ledger keepers: they maintain transaction records, verify account ownership (KYC — Know Your Customer), validate transactions, and adjudicate disputes.
- Identity verification is central to this system: banks link faces to documents (e.g., photo with ID) to establish who holds an account.
- Transaction validity in the traditional system means having sufficient funds or an approved credit/loan arrangement.
- The entire system ultimately requires trusting the central bank as the final authority on monetary value and transaction legitimacy.

3. Bitcoin's Origin and the Motivation for Decentralization

Bitcoin was created specifically to remove centralized control over money production and distribution, introducing the concept of a distributed ledger where no single authority can unilaterally determine value, control supply, or adjudicate transactions.

- Bitcoin inaugurated the concept of a distributed ledger and was motivated by the “cypherpunk” philosophy: remove governments and central banks from the monetary system.
- The key innovation: replace central authority with a protocol — a set of rules everyone in the distributed system follows, with no single participant able to deviate unilaterally.
- Three fundamental problems that decentralization must solve: (1) identity — how to know who is who without a central registry; (2) integrity — how to ensure transaction history cannot be reordered or altered; (3) ownership — how to ensure only the rightful owner can spend funds.
- “Trustless” does not mean trust is absent; it means trust is placed in the cryptographic protocol rather than in any specific person or institution.

4. Cryptographic Primitives as the Foundation of Blockchain

Four primary cryptographic primitives — hash functions, digital signatures, randomness, and consensus — collectively solve the problems of integrity, ownership, and manipulation resistance in decentralized systems. The failure of any single primitive compromises the entire blockchain.

- Cryptographic hash functions provide **integrity**: blocks in a blockchain are chained together via hashes; if hashes are broken, the chain breaks.
- Digital signatures provide **ownership**: the private key controlling a wallet is the digital signature mechanism; if broken, wallets can be stolen.
- Randomness provides **key security and entropy**: good key generation and certain consensus decisions require high-quality randomness.
- Consensus ensures all participants share the same copy of the distributed ledger, updated to the same state — analogous to everyone reading the same page of the same book.
- Before blockchains: trust the bank. After blockchains: trust SHA-256 (or equivalent). If SHA-256 is broken, most blockchains fail — but breaking a mathematical algorithm may be harder than corrupting a human institution.

5. Mathematical Hardness Problems and the Quantum Computing Threat

The security of current cryptographic systems — both internet-wide (RSA/prime factorization) and blockchain-specific (elliptic curve cryptography) — rests on mathematical problems that are computationally infeasible for classical computers but solvable by sufficiently powerful quantum computers, representing an existential threat to existing cryptographic infrastructure.

- 70–80% of internet security today is based on the hardness of prime factorization: multiplying two large primes (600+ digits each) is trivial; factoring the result is computationally intractable for classical computers.
- Quantum computers can solve prime factorization in a single step using superposition, given sufficient qubits — breaking RSA-based cryptography.
- Elliptic curve cryptography (used in most blockchains) is similarly vulnerable to quantum attacks; blockchain projects must eventually migrate to quantum-resistant algorithms.
- A “cryptographically relevant” quantum computer is the actual threat milestone — current machines are not powerful enough to break RSA; breaking RSA will be followed shortly by breaking elliptic curve cryptography using the same threshold machine.
- Crypto agility — plugging and unplugging different cryptographic algorithms — is being implemented now to prepare systems for the transition.
- Elliptic curve cryptography has 45 years of study; post-quantum cryptography has only ~10 years and may have undiscovered design flaws.
- Government transition roadmap: post-quantum PKI by 2027, all government systems by 2030, RSA/ECDSA retired by 2035.
- Hash functions are significantly more quantum-resistant: collision space drops from 2^{256} to 2^{128} , still considered very safe.
- Some post-quantum digital signature schemes are hash-based, leveraging the resilience of hash functions.
- Some mathematical problems do resist quantum attacks; research into post-quantum cryptography is identifying these alternatives.

6. Cryptographic Hash Functions — Properties and Security Model

A cryptographic hash function maps an arbitrarily large (potentially infinite) input domain to a fixed-size output, making it computationally infeasible to reverse the function. The three critical security properties — pre-image resistance, second preimage resistance, and collision resistance — together enable tamper detection and form the basis for blockchain integrity and mining.

- A hash function is formally a function: it maps every element of a domain to an element of a codomain (injective mapping concept).
- A **cryptographic** hash function specifically maps an **arbitrary-size** input to a **fixed-size** output — mapping a potentially infinite set into a finite set.
- Because the output space is finite and the input space is (potentially) infinite, collisions — multiple inputs mapping to the same output — are inevitable; however, collisions are actually useful for security because the function is not invertible.
- Inputs larger than the hash block size are handled by hashing the first part, then concatenating that hash as the prefix of the next segment (chained hashing).
- Most hash functions use linear congruence schemes — spreading input into registers and repeatedly squeezing with XOR — making them extremely efficient for hardware implementation.
- The avalanche effect means changing even one ASCII character (1 bit) in the input produces a completely different and unpredictable output hash; a good hash function changes at least 50% of output bits when the input changes by 1 bit.
- Hash functions are designed to maximize diffusion (spreading input bits across registers) and confusion (obscuring the relationship between input and output).
- SHA-256, for example, maintains eight 32-bit internal state registers, each updated iteratively by every input block.
- **Pre-image resistance:** Given a hash output, it is computationally infeasible to find the original input — this property underpins blockchain mining (proof-of-work).
- **Second preimage resistance:** Knowing that input A maps to hash H, it is computationally infeasible to find a different input B that also maps to H — this protects against forged signatures and document substitution attacks.
- **Collision resistance:** It is computationally infeasible to find any two distinct inputs that produce the same hash output — the hardest property to guarantee.
- The birthday paradox illustrates why collision resistance is statistically difficult: in a group of 50 people, there is a 50% chance two share a birthday because the date is not fixed — analogously, collision attacks don't target a specific hash.
- When all three properties hold, the hash provides full **tamper detection**: any modification to data breaks the hash, revealing manipulation.

7. Hashes as the Backbone of Blockchain Structure

A blockchain is a distributed digital ledger — historically analogous to a six-thousand-year-old accounting book — where each block (page) contains transactions and the hash of the previous block. Hashes chain the blocks together and guarantee immutability: changing any bit in any block invalidates all subsequent hashes, making tampering immediately detectable by anyone holding the chain.

- A ledger is a 6,000-year-old accounting technology used to record transactions; blockchain is its digital, distributed, decentralized equivalent.
- Each block in a blockchain corresponds to one page of the ledger, containing a timestamp, transaction data, and the hash of the previous block.
- A blockchain is structurally a linked list with integrity control: instead of only a pointer to the next/previous element, each node holds a cryptographic hash ensuring the data hasn't changed.
- The genesis block (head of the list) anchors integrity — holding just its hash allows verification of the entire chain by traversal.
- If anyone alters even one bit in any block, the hash of that block changes, which breaks the hash reference in the next block, propagating a detectable break through the entire chain.
- Multiple synchronized, distributed copies of the blockchain exist — no single central authority owns or controls the ledger.

8. Merkle Trees and Transaction Verification

The Merkle tree is a binary hash tree used in blockchains to organize block transactions, enabling integrity verification in $O(\log n)$ time rather than $O(n)$, by storing transaction hashes at the leaves and combining hashes upward to a single root.

- All transactions in a block are treated as simultaneous (at “block time”) for verification purposes; a transaction can only spend outputs from previous blocks, not the same block.
- Leaves of the Merkle tree contain hashes of individual transactions; internal nodes contain the hash of the concatenation of their two children's hashes.
- The root hash summarizes the entire transaction dataset; holding the root allows integrity verification of any transaction in $O(\log n)$ by accessing only sibling nodes (“Merkle proofs” or “inclusion proofs”).
- Verification complexity is $O(\log n)$ vs. $O(n)$ for linear search — a significant efficiency gain.

9. Proof-of-Work, Block Structure, and Consensus

Proof-of-Work mining uses the pre-image resistance and time-determinism of hash functions to create a fair computational puzzle. Miners repeatedly hash a block with varying nonce values until the resulting hash meets a target condition (required number of leading zeros). The network self-calibrates difficulty to maintain a ~10-minute block interval, and consensus is majority-based — the longest chain wins in the event of a fork.

- A Bitcoin block contains: hash of the previous block, timestamp, miner reward address, nonce, and a Merkle tree of transactions.
- Mining is the process of changing the nonce and hashing the entire block repeatedly until the hash meets the network's current target (required leading zeros); this is a direct application of pre-image resistance.
- The difficulty of mining is calibrated by adjusting the required number of leading zeros — more zeros means a smaller valid output set, requiring more attempts on average.
- Before including a transaction, miners must verify it references unspent outputs to prevent double-spending.
- The network adjusts difficulty based on the mean time between the last ~100 blocks, targeting a 10-minute block interval.
- When a valid block is found, the miner broadcasts it to the network and receives a cryptocurrency reward (currently 3.125 BTC per block) plus transaction fees embedded in the block.
- Hash functions are time-deterministic: the same input always computes in the same number of instructions, so speed advantage comes only from faster hardware — making the puzzle objectively fair relative to computational resources.
- When a valid block is broadcast, other miners verify it by hashing it and checking the result; if valid, they add it to their chain and immediately begin mining the next block.
- In the event of a network split (fork), the longest chain (most accumulated work) wins, and miners on the shorter chain must discard their alternate-chain blocks.
- Cryptocurrencies use pseudonymity — users can create unlimited identities — making vote-based consensus unfair; proof-of-work shifts the fairness criterion to computational power instead.

10. Authorization and Digital Signatures

Authorization in blockchain mirrors the concept of signing a transaction in traditional finance, but is implemented digitally. A digital signature provides authentication, integrity, and — critically — non-repudiation,

proving that a specific private key holder authorized a transaction and cannot later deny it.

- In traditional banking, authorization uses usernames/passwords, cards, biometrics, or PINs — all forms of proving identity to allow a transaction.
- A digital signature is conceptually analogous to a handwritten signature on a check: it proves the signer's identity and agreement, but is now performed cryptographically.
- Digital signatures provide three properties: **authentication** (confirming identity), **integrity** (the message was not altered), and **non-repudiation** (the signer cannot deny having signed).
- Every blockchain transaction must be signed, and the validity of a signature can be checked by anyone with the corresponding public key — no trusted third party is required.

11. Symmetric vs. Asymmetric Cryptography

Cryptography evolved from ancient substitution ciphers to symmetric (single-key) cryptography, with its key-exchange limitation, to asymmetric (public/private key) cryptography discovered publicly in 1977 (and privately by British intelligence in 1972), which eliminates the need to pre-share secrets.

- Ancient cryptography (Rome, Greece) used ciphers so that intercepted messages could not be read by unauthorized parties.
- Symmetric cryptography uses one shared secret to both encrypt and decrypt; its major limitation is the need to securely meet and share the key in advance.
- Alan Turing's foundational work in computing — including the theory of computation — was developed primarily to break World War II cryptography (Enigma).
- Asymmetric cryptography uses two mathematically linked keys: a public key (shareable with anyone) and a private key (kept secret). Data encrypted with one key can only be decrypted with the other.
- Encrypting with a **public key** → only the private key holder can decrypt (confidentiality). Encrypting with a **private key** → anyone with the public key can verify it came from the private key holder (digital signature).
- The direction of encryption determines whether you achieve *confidentiality* (public key encrypts) or *authentication/non-repudiation* (private key signs).

12. Blockchain Identity: Accounts, Wallets, and Ownership

In blockchain, identity is not tied to a name or institution but solely to possession of a private key. The public key serves as the account address,

and the private key (the “wallet”) is the only means of authorizing spending. Losing the private key means losing access to the funds permanently.

- In blockchain terminology: **public key = account, private key = wallet.**
- To receive funds, you share your public key (account). To spend funds, you must sign a transaction with the corresponding private key.
- Nobody on the blockchain knows who you are — your only provable identity is control of the private key.
- Ownership is defined as the ability to sign with the private key corresponding to the public key that holds the funds.
- There is no central recovery mechanism in blockchain — if you lose the private key, the funds are gone permanently.

13. ECDSA and Elliptic Curve Cryptography

ECDSA (Elliptic Curve Digital Signature Algorithm) is the standard signing algorithm in blockchains. It replaces the large natural-number prime arithmetic of DSA/RSA with elliptic curve point arithmetic, achieving equivalent or superior security with dramatically smaller (256-bit) keys, and produces uniformly 32-byte data structures that are highly efficient for computer operations.

- DSA (Digital Signature Algorithm), published by NIST, uses a random number combined with the private key to produce a signature verifiable with the public key; signing the same message twice yields two different signatures due to randomness.
- RSA/DSA require very large primes (e.g., 2^{2048}) because primality density among natural numbers is low (~14–18% up to 100,000, decreasing further for large numbers).
- Elliptic curves are defined by equations of the form $y^2 = x^3 + ax + b$; addition and multiplication of points on the curve always yield another point on the curve.
- “Prime points” on elliptic curves have analogous properties to prime numbers, but primality density among elliptic curve points is ~60%, allowing much smaller key sizes for equivalent security.
- Operations on elliptic curves use XOR and AND rather than modular exponentiation, making them very fast.
- Bitcoin uses a specific elliptic curve (secp256k1); the private key is a random 256-bit number; the public key is a point on the curve.
- 256-bit keys and SHA-256 hashes both produce 32-byte values, making all core blockchain data structures (account, wallet, block hash, transaction ID)

uniformly 32 bytes — aligning with computer word sizes for maximum efficiency.

- ECDSA's discrete logarithm problem is more resistant to quantum attacks than RSA's prime factorization problem, requiring more qubits to break.
- The 256-bit key size in ECDSA is not weaker than RSA-2048 — the underlying mathematical problem (elliptic curve discrete logarithm) is harder to solve, including for quantum computers.

14. Blockchain Wallets, Private Keys, and Self-Custody

A blockchain wallet is a control mechanism, not an asset container — the blockchain records balances, and the private key grants spending control. Losing the private key results in irreversible loss of funds, with no central authority to appeal to. Self-custody is technically complex and risky; most users should use custodial services instead.

- The wallet does not store assets; assets are recorded on the blockchain at a public key address.
- Self-custody means managing your own private keys; it is powerful but extremely risky and complex — it is recommended that most users avoid it unless they are highly disciplined and prepared.
- Jean Martina's self-custody method: generated 20 paper wallets offline (Linux USB boot, JavaScript software), printed 3 password-encrypted copies of each, stored with himself, his parents, and a lawyer.
- Spreading funds across multiple wallets reduces the risk of total loss from a single forgotten password or lost copy.
- If held on an exchange, a court order can compel the exchange to seize funds; self-custody prevents this but transfers all risk to the individual.
- The 2017 Bitcoin forks (Bitcoin Cash, Bitcoin Gold) were a motivating event for Jean Martina to move to self-custody — exchanges, not users, controlled fork distributions.

15. Seed Phrases and Hierarchical Deterministic Wallets

Seed phrases (BIP32/BIP39 standards) offer a convenient alternative to managing individual keys by deriving all private and public keys from a single master secret. However, they introduce a single point of failure at the seed level and require very high entropy passphrases to avoid brute-force vulnerabilities.

- A seed phrase is a master password from which all private/public key pairs can be deterministically derived.
- Users only need to remember the seed and the derivation path (e.g., "tenth wallet of this seed") to regenerate all keys.

- BIP32 and BIP39 are the relevant standards governing hierarchical deterministic wallets; hardware wallets use seed phrases as their underlying key management mechanism.
- If the seed is compromised, all derived keys are compromised — it is a single point of failure.
- Seed passphrases must have entropy equivalent to a random key (2^{255} possible values) to resist brute-force attacks; human-chosen passphrases using dictionary words, names, or predictable patterns have far lower entropy and are vulnerable to targeted attacks.

16. Randomness, Entropy, and Cryptographic Security

Good randomness is the cornerstone of cryptographic security. Both TRNGs and PRNGs play complementary roles — TRNGs provide a high-quality seed of true entropy, and PRNGs expand that seed into usable cryptographic material. Weak entropy has caused real-world failures in voting machines, Bitcoin wallets, and the PlayStation Network.

- Cryptographic systems are deterministic computers attempting to simulate true randomness, which is fundamentally difficult.
- TRNGs use physical phenomena (e.g., reverse-biased transistors producing static noise) to generate unpredictable output.
- PRNGs take a fixed input (seed) and output something that looks random but is not truly random; TRNGs are used to seed PRNGs because TRNGs produce very little entropy but of very high quality.
- Bad entropy reduces the key space, making cryptographic keys predictable and vulnerable to brute-force attacks.
- A real-world example: Brazilian voting machines were seeded with the exact timestamp printed on the machine's startup slip ("zerésima"), allowing an attacker to reconstruct all shuffle operations and de-anonymize votes.
- Sony's PlayStation Network suffered from bad/reused entropy in its cryptographic implementation, enabling game cloning.
- Reusing nonces in ECDSA due to weak entropy allows private key recovery by attackers.

17. Zero-Knowledge Proofs and Blockchain Privacy

Bitcoin's public transaction model means wallet balances and transaction histories are traceable, posing significant privacy risks. Zero-knowledge proofs (ZKPs) allow one party to prove a statement is true without revealing the underlying data. ZKPs are increasingly being integrated into blockchains and digital identity systems and are expected to become a mandatory feature within approximately five years.

- In Bitcoin, every transaction is publicly recorded; knowing someone's public address allows anyone to determine their full transaction history and balance.
- The goal of ZKP integration is to hide transaction value, origin, and destination on public blockchains.
- Traditional cryptographic proofs require revealing data; ZKPs allow proving a fact without exposing the data itself.
- A practical use case: proving you are over 18 years old to a club without revealing your name, address, date of birth, or national ID number — only a government-signed proof of the age assertion is shared.
- Brazil passed a law requiring operating systems to identify underage users, highlighting the real-world urgency of privacy-preserving age verification.
- ZKPs are not required for a basic blockchain to function but are critical for privacy-preserving systems; the lecturer predicts ZKPs will be a mandatory feature of most blockchains within five years.

18. Broader Blockchain Architecture and Data Structures

Blockchain is not a single technology but a composable system of cryptographic primitives, each solving a distinct problem. Beyond the classic linked-list blockchain, alternative data structures such as Directed Acyclic Graphs (DAGs/hash graphs) offer higher throughput, and various Merkle tree variants optimize for different blockchain characteristics.

- Blockchain should be understood as a composition of primitives addressing integrity, ownership, randomness, and agreement.
- Distributed Ledger Technology (DLT) is not limited to linked-list structures; hash graphs use Directed Acyclic Graphs (DAGs) with multiple tips that can merge, enabling much greater throughput and speed.
- Merkle trees are the basic internal data structure for transactions, but more optimized variants exist for specific blockchain use cases.
- Understanding the role of each primitive is essential to understanding how any blockchain is built.

Suggestions

- If you are interested in understanding the full mathematical depth behind these cryptographic primitives, take a dedicated computer security course — it will cover these same primitives in detail over roughly a month and a half.
- When reasoning about the security of blockchain systems, always consider what happens if a core cryptographic primitive fails — this systems-level

thinking (e.g., “if SHA-256 breaks, what breaks with it?”) is essential for evaluating real-world blockchain robustness.

- When studying blockchain, always ground your understanding in the linked list data structure first — blockchain is a linked list with cryptographic integrity control, and this framing makes the concept immediately accessible to anyone with basic computer science knowledge.
- Do not conflate “collisions existing” with “collision attacks being feasible” — the distinction between inevitable collisions (birthday paradox) and engineered second-preimage attacks is the key to understanding why hash security is nuanced and property-dependent.
- Understand that every blockchain transaction must be signed with a private key, and that signature validity can be verified by anyone with the corresponding public key — no trusted third party is needed; internalize the asymmetry: the direction of encryption determines whether you achieve *confidentiality* (public key encrypts) or *authentication/non-repudiation* (private key signs).
- Remember that the 256-bit key size in ECDSA is not weaker than RSA-2048 — the underlying mathematical problem (elliptic curve discrete logarithm) is harder to solve, including for quantum computers.
- Consider the quantum computing threat proactively: blockchain projects relying on elliptic curve cryptography will need to migrate to quantum-resistant schemes; begin understanding the post-quantum cryptography transition timeline (2027–2035) now, especially if working with government systems, PKI, or long-lived data requiring confidentiality.
- Consider crypto agility when designing or evaluating any security-related system — the ability to swap cryptographic algorithms will be essential in the coming decade.
- Avoid self-custody of cryptocurrency unless you are highly disciplined, technically prepared, and have a robust physical backup system (multiple encrypted copies in geographically and legally separated locations). If you do practice self-custody, never put all funds in a single wallet.
- When choosing a seed phrase or passphrase, prioritize high entropy — avoid dictionary words, personal names, or any predictable patterns, as these dramatically reduce security.
- Treat randomness as seriously as algorithm choice: always use a properly seeded, high-entropy source for any cryptographic application, and never seed a PRNG with low-entropy data such as timestamps or system clocks.
- When designing or auditing systems that use cryptography, evaluate each primitive independently — hashing, signatures, and randomness — because the overall system is only as secure as its weakest component.

- Consider integrating zero-knowledge proofs into any blockchain or identity system where privacy of transaction value, origin, destination, or personal attributes is a requirement.
- When building or evaluating blockchain systems, always verify that transactions reference unspent outputs (UTXO model) before inclusion in a block to prevent double-spending.
- When evaluating trust in any financial or distributed system, ask not just “who do I trust?” but “what do I trust, and what are the failure modes of that thing?” — shifting trust from people to mathematics changes the risk profile fundamentally.

AI Suggestions

- The core of this lecture series spans five interconnected topics. It is recommended to work through each hands-on exercise in sequence:
 - Cryptographic Hash Functions:** Take several different inputs (a word, a sentence, a file) and run them through SHA-256 using a free online tool or a simple Python script (`hashlib`) — observe fixed-size outputs, the avalanche effect (tiny input change → completely different hash), and the impossibility of reversing the output.
 - Three Security Properties of Hash Functions:** Compute a SHA-256 hash, change one character, recompute, and observe the avalanche effect. Then articulate in your own words why each of the three properties (pre-image resistance, second preimage resistance, collision resistance) matters for blockchain integrity.
 - Asymmetric Cryptography and Digital Signatures:** Use a tool like OpenSSL or a Bitcoin wallet library (e.g., `bitcoinlib` in Python) to generate a public/private key pair, sign a sample message with the private key, and verify it with the public key — observe that the same message signed twice produces two different ECDSA signatures (due to randomness), while both verify correctly.
 - Private Key Ownership and Irreversibility:** Set up a test wallet on a blockchain testnet, generate a key pair, deliberately delete the wallet software, and attempt to recover funds using only your private key or seed phrase — experience the recovery (or non-recovery) process in a low-stakes environment.
 - Randomness and Entropy:** Implement a PRNG seeded with a fixed timestamp and observe how the output is fully reproducible, then contrast it with output seeded from a TRNG source (e.g., using Python’s `os.urandom()` vs. `random.seed(fixed_value)`).
- **Extracurricular Resources:**
 - [3Blue1Brown — “But how does bitcoin actually work?” (visual, intuitive explanation of hashing and blockchain structure):

<https://www.youtube.com/watch?v=bBC-nXj3Ng4>]

- [Computerphile — “SHA: Secure Hashing Algorithm” (clear walkthrough of how SHA-256 works):
<https://www.youtube.com/watch?v=DMtFhACPnTY>]
- [Computerphile — “Public Key Cryptography” (accessible visual explanation of asymmetric cryptography):
https://www.youtube.com/watch?v=GSIDS_lvRv4]
- [Computerphile — “Random Numbers with COMPUTERPHILE” (why randomness is hard for computers and how it is used in cryptography): <https://www.youtube.com/watch?v=1cUUfMeOijg>]
- [Khan Academy — “Cryptography” course (covers hashing, prime factorization, and public-key cryptography from first principles):
<https://www.khanacademy.org/computing/computer-science/cryptography>]
- [Khan Academy — Cryptographic Hash Functions (beginner-friendly video explanation):
<https://www.khanacademy.org/computing/computer-science/cryptography/modern-crypt/v/cryptographic-hash-functions>]
- [SHA-256 Interactive Hash Tool — compute and visualize hashes in your browser: <https://emn178.github.io/online-tools/sha256.html>]
- [Bitcoin Wiki — Proof of Work (explains how hash-based mining and the nonce mechanism work in practice):
https://en.bitcoin.it/wiki/Proof_of_work]
- [Bitcoin Wiki — Elliptic Curve Digital Signature Algorithm (ECDSA) explained with blockchain context:
https://en.bitcoin.it/wiki/Elliptic_Curve_Digital_Signature_Algorithm]
- [Andrea Corbellini — “Elliptic Curve Cryptography: a gentle introduction” (detailed interactive blog post):
<https://andrea.corbellini.name/2015/05/17/elliptic-curve-cryptography-a-gentle-introduction/>]
- [Bitcoin Improvement Proposals BIP32 & BIP39 — Official specifications for hierarchical deterministic wallets and mnemonic seed phrases: <https://github.com/bitcoin/bips>]
- [NIST Post-Quantum Cryptography Standardization — Official resource tracking the U.S. government’s post-quantum algorithm selection and standards: <https://csrc.nist.gov/projects/post-quantum-cryptography>]
- [NIST Special Publication 800-90A — Recommendation for Random Number Generation Using Deterministic Random Bit Generators:
<https://csrc.nist.gov/publications/detail/sp/800-90a/rev-1/final>]

- [Mastering Bitcoin by Andreas M. Antonopoulos (free online) — In-depth technical coverage of keys, wallets, and blockchain security fundamentals: <https://github.com/bitcoinbook/bitcoinbook>]
- [[random.org](https://www.random.org) — Introduction to Randomness and Random Numbers (beginner-friendly explanation of true vs. pseudo-randomness): <https://www.random.org/randomness/>]